DHAR

Bare Metal. Absolute Control.

VOLUME II — ENGINEERING ROADMAP

STAGE-DELTA

"Gearing Dhar — and Kun: declare what should exist, and it does."

DOCUMENT	Stage-delta — Gearing Dhar
VERSION	2.2
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume II
DEPENDS ON	Spec0 §0 (naming), Spec1 §13 (Collections), §18 (foreign), Amendment A-1
PRECEDES	Stage-epsilon — Deploying Dhar

Contents

1. Mandate 3

2. Standard Library 3

2.1 Domain modules 3

3. Kun — The Package Manager 5

3.1 Why “Kun” 5

3.2 Project manifest 5

3.3 Dependency resolution & lockfiles 5

3.4 Versioning 5

3.5 Workspaces 6

3.6 Registry & publishing 6

3.7 Security & provenance 6

3.8 Test oracles are not dependencies 6

4. Interop & the Foreign Ecosystem 6

5. Developer Tooling 7

6. Required Architecture 7

7. Acceptance Criteria 7

8. Handoff to Stage-epsilon 8

1. Mandate

Give Dhar a standard library, a named and fully specified package manager, and a developer toolchain good enough that writing real software in it does not require reinventing basic infrastructure — and give the collection types Spec1 §13 already fixed their real implementations.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). Stage-delta is where that becomes a library. The three tracks each get the modules their gates need, and every other domain Dhar serves — systems and hardware, web and backend, data, blockchain and Web3 — is delivered here too, as modules on the same core, so that Dhar stays modular and able to do what its users need.

2. Standard Library

MODULE	COVERS
std.io	Console and file input/output, built on gen/sys (Spec1 §20)
std.mem	Arena helpers and manual-escape-hatch allocation utilities
std.collections	list, set, dict, tuple support code, plus Stack/Queue/Deque (Spec1 §13.7)
std.fmt	String formatting, building on f-string interpolation (Spec1 §2.4)
std.net	Networking
std.fs	Filesystem access
std.crypto	Cryptographic primitives, classical and post-quantum (§2.1)
std.math	Numeric functions beyond the core operators (Spec1 §6)
std.num	Helpers for the numeric types of Spec1 §4.7: conversions, fixed-point, the wrapping, saturating and checked variants (§2.1)
std.tensor	Fixed-shape arrays, checked views and arena-backed tensors (Spec1 §14.6; §2.1)
std.quantum	Quantum circuits as data, OpenQASM 3 output, a state-vector simulator (§2.1)
std.concurrent	Higher-level concurrency utilities built on spark/draw (Spec1 §15)

2.1 Domain modules

The standard library and Kun deliver the domains as modules on the same core, held to the same safety floor (Amendment A-1, P2 and P6). The order is the order of the tracks: cybersecurity, artificial intelligence, quantum computing, then the rest. Outside implementations (OpenSSL, libsodium, NumPy, Qiskit and the like) are test oracles

only, never dependencies (§3.8). None of the track modules begins before Stage-beta compiles itself (Amendment A-1 §7). A module outside the three tracks has no gate yet; a later amendment may add one, and such a module is never held back because no gate has been written for it.

DOMAIN	MODULE	CONTAINS	GATE
Cybersecurity	`std.crypto`	SHA-256 (FIPS 180-4); SHA-3 and SHAKE (FIPS 202); ChaCha20-Poly1305 (RFC 8439); X25519 (RFC 7748); Ed25519 (RFC 8032). Every secret-handling routine is written with secret values (Spec1 §14.5) and has a constant-time comparison.	S3: published known-answer vectors, and a timing test for each secret-handling routine
Cybersecurity	`std.crypto`, post-quantum	ML-KEM (FIPS 203), ML-DSA (FIPS 204), SLH-DSA (FIPS 205). Needs no quantum hardware; uses SHA-3 and SHAKE, so it can be scheduled as soon as S3 passes.	Q1: NIST known-answer tests
Cybersecurity	Security tooling (Kun packages)	Parsers for hostile input, protocol libraries, forensic and defensive tools, written in the checked style of Spec1 §14.	S1, the language's memory-safety floor; no module gate
Artificial intelligence	`std.num`	The numeric types of Spec1 §4.7 for models: `f32`, `f64`, `f16`, `bf16`, fixed-point, 8-bit integers, fixed-width vectors.	A1: bit-identical results on edge values
Artificial intelligence	`std.tensor`	Fixed-shape arrays and checked views, arena-backed so that a model's whole memory use is one allocation known before it runs (Spec1 §14.6); the kernels the A4 benchmark needs, matrix multiplication first.	A2: out-of-range and misused views never corrupt memory
Artificial intelligence	Inference (a Kun package)	Reads a documented, simple weight-file format and runs a small trained network (a handwritten-digit classifier is the reference) with no outside runtime.	A3: predictions agree with a reference framework's to a stated tolerance; A4 measures the speed (Stage-gamma §1)
Quantum computing	`std.quantum`	A quantum circuit (qubit registers, gates, measurement, classical control) as an ordinary value, written out as OpenQASM 3; a state-vector simulator on `c128` that states its qubit limit and refuses beyond it.	Q2: a reference tool accepts and simulates the files; Q3: amplitudes agree with a reference simulator to a stated tolerance
Systems and hardware	`std.io`, `std.mem`, `std.fs`; raw memory (Spec1 §14.4)	Console and file input/output, arena helpers, volatile access to hardware registers, the system interface on every committed target.	None yet; the safety floor applies
Web and backend	`std.net`, `std.fmt`, Kun packages	Networking and the web and backend libraries built on it. WebAssembly is a committed target (Amendment A-2 §3.1), so Dhar programs also run in a browser.	None yet

DOMAIN	MODULE	CONTAINS	GATE
Data	`std.collections`, `std.fmt`, Kun packages	The collection types of Spec1 §13 and the libraries for data formats and processing built on them.	None yet
Blockchain and Web3	Kun packages	Libraries built on `std.crypto` for hashing, signatures and canonical serialisation; deterministic, bounded-memory programs, with the WebAssembly target as a deployment form for runtimes that execute WebAssembly.	None yet

Every library's conformance vectors are its own tests, run by `dhar test` (§5).

3. Kun — The Package Manager

3.1 Why “Kun”

Named for the Arabic *kun*, “be” — the imperative in the Quranic *kun fa-yakun*, “be, and it is” (Spec0 §0). A package manager's core promise is the same shape at a vastly smaller scale: declare what a project needs, and it comes into being — resolved, fetched, verified, ready — without being assembled by hand. The name is a mandate as much as a label: everything in this section exists to make that declaration-to-existence promise actually true, not just poetic.

3.2 Project manifest

Every Kun-managed project declares, in one manifest file: project name and semantic version; dependencies, each with a version constraint; the minimum Dhar language version required (Spec1's own version, per its revision log); and build target(s) (Spec0 §4's backend list).

3.3 Dependency resolution & lockfiles

1. Kun must resolve a full dependency graph deterministically: the same manifest must always produce the same resolution. This is not a quality goal, it is a direct precondition for Stage-epsilon's reproducible-builds requirement — a build cannot be reproducible if its own dependency resolution is not.
2. Resolution produces a lockfile recording every resolved version and its content hash. A second kun install against an unchanged manifest must reuse the lockfile exactly, never silently re-resolve to different versions, and must do so entirely from local cache with no network round-trip required — a design requirement, not an optimization, since Casting Dhar (Volume III) documents real, widespread connectivity constraints across Dhar's primary target geography that a network-dependent install step would fail against.
3. A version conflict that cannot be resolved must fail loudly, with the conflicting requirements shown, rather than silently selecting one and hoping.

3.4 Versioning

Kun enforces semantic versioning (MAJOR.MINOR.PATCH) for anything published to the public registry: a breaking change requires a MAJOR bump, checked where practical against the package's own declared bond (Spec1 §12) and mold field signatures, not merely taken on the publisher's word.

3.5 Workspaces

A single manifest may declare multiple local packages sharing one lockfile and one resolution — required for a project shaped like Dhar's own (compiler, standard library, and tooling as separate packages built together), and for any real application split into internal libraries rather than one monolith.

3.6 Registry & publishing

Kun **must** support a public registry: name reservation, and immutable published versions — once published, a version's content cannot change, only be yanked. Kun **must also** support pulling from a private, self-hosted registry as a first-class configuration, not an afterthought: an institution building critical infrastructure (Spec0 §5) cannot make its supply chain depend on a single externally controlled registry, which would quietly reintroduce the exact dependency Spec0 §2.1 exists to remove.

3.7 Security & provenance

- Every published package carries a content hash and a signature, verified on install.
- Kun must be able to emit an SBOM entry for every resolved dependency, feeding directly into Stage-epsilon's SBOM requirement — a release's software bill of materials is only as complete as its package manager's records.
- A yanked version must still be resolvable for existing lockfiles (so old builds keep building) but must not be selected for any new resolution.

3.8 Test oracles are not dependencies

Amendment A-1, P1: the core language, the compiler and the standard library depend on no outside compiler, runtime or library. An outside implementation may be used to check Dhar's answers, never to produce them. Kun therefore has a separate class of **oracle dependency**: it is available to tests and benchmarks only, is never linked into a build of the core, the standard library or a release, never appears in a release's resolved dependencies, and is listed separately in the bill of materials. Kun refuses a release in which an oracle dependency has reached the build.

4. Interop & the Foreign Ecosystem

Dhar's own ecosystem, distributed through Kun, is necessary but not sufficient. Real adoption requires that Dhar code can call into, and be called from, the existing ecosystems of C, C++, Rust, Java, and Python. Spec1 §18 specifies `foreign`, the single interop primitive everything below rests on: a Dhar declaration of an external, C-ABI-compatible function. Every language here is reachable through that one mechanism, though the directness of the path varies — and this document says so honestly rather than promising uniform ease that would not survive contact with reality.

LANGUAGE	PATH	DIRECTNESS
C	foreign declares the function directly; C's ABI is exactly what foreign targets.	Direct
C++	A thin extern “C” wrapper compiled alongside the C++ library, then targeted by foreign — the same solution every language uses, since C++ has no single cross-compiler ABI to target directly.	Indirect, by necessity
Rust	A Rust function marked #[no_mangle] extern “C” exposes a stable C boundary — already common practice via -sys/-ffi crates — which foreign then targets.	Indirect, standard practice

LANGUAGE	PATH	DIRECTNESS
Java	A JNI (Java Native Interface) bridge, the same mechanism every non-JVM language uses to reach the JVM; foreign targets the bridge's C side.	Indirect, requires a shim
Python	Two directions: Dhar calling Python embeds CPython via its C API (Python.h) through foreign; Python calling Dhar loads a Dhar shared library (built from expose-d functions) via ctypes/cffi, with no change needed on the Dhar side.	Indirect one way, near-direct the other

Kun's role in all five: fetching and building the wrapper/shim crates each indirect path needs is an ordinary dependency resolution, not a special case, and generating the JNI boilerplate automatically is a strong candidate for Stage-delta-and-later tooling work rather than a core-language feature.

The common thread is deliberate: one interop keyword, targeting the one interop layer every mature ecosystem already knows how to speak, directly or through a thin, well-established bridge. That is more auditable than five different interop mechanisms each claiming a seamlessness none of them fully deliver — and more honest about where the real friction actually is.

5. Developer Tooling

- `dhar build` — builds a multi-file, multi-module, multi-package (workspace-aware) project from one command.
- `dhar test` — discovers and runs tests without per-file configuration.
- `dhar fmt` — the canonical formatter, resolving any style question Spec1 leaves to convention.
- A documentation generator, producing reference docs from source comments.
- LSP support: diagnostics and go-to-definition at minimum.

This list is the inventory of what the standard distribution contains. The requirements for these tools, for the further commands (`dhar test --vectors` , `dhar verify-build` , `dhar sbom`), for the quality of diagnostics and for the in-browser playground are Tooling Dhar's (Volume III; the playground is Tooling Dhar §5).

6. Required Architecture

The standard library is written in Dhar itself, using Stage-gamma's compiler, and follows the same one-definition-per-file discipline as every earlier stage. Kun is written in Dhar as well, and is itself the first real test of the standard library's std.net and std.fs modules under load. Compiler intrinsics are permitted only where Spec1's memory model (§14) or performance genuinely requires them; everything else **must** be ordinary Dhar code, per the design stance Spec1 §13 already commits to. The track modules of §2.1 are written with the secret-value rules of Spec1 §14.5 on every secret-handling routine, and are not begun before Stage-beta compiles itself (Amendment A-1 §7).

7. Acceptance Criteria

1. Every collection type in Spec1 §13 has a working implementation matching its specified literal syntax exactly.

2. Kun resolves a real multi-package workspace deterministically, and a second install against an unchanged manifest reuses the lockfile without re-resolving.
3. Kun can publish to and install from both a public registry and a self-hosted private one, with no code path that assumes only the former exists.
4. At least one working interop example exists for each of C, C++, Rust, Java, and Python, per §4's table — not merely specified, demonstrated.
5. dhar build builds a real multi-file, multi-module project from a single command; dhar test discovers and runs that project's tests with zero per-file configuration.
6. LSP support is verified against at least one real editor integration.
7. `std.crypto` passes gate S3: the published known-answer vectors for every algorithm of §2.1, and for each secret-handling routine the timing test of Amendment A-1 §9 (two input classes, at least a million runs each, a published statistical method) showing no significant difference.
8. The post-quantum algorithms pass the NIST known-answer tests (gate Q1), after S3.
9. `std.num` and `std.tensor` pass gates A1 and A2; the inference example passes A3.
10. `std.quantum` passes gates Q2 and Q3, including the refusal beyond its stated qubit limit.
11. Every library's conformance vectors run under `dhar test`; no release contains an oracle dependency (§3.8).
12. Each module outside the three tracks has tests and keeps the safety floor of Spec1 §14; none is held back because no gate has been written for it (Amendment A-1, P6).

8. Handoff to Stage-epsilon

Stage-epsilon assumes a working build system and a working, secured package manager already exist; it hardens what this stage builds — including consuming Kun's SBOM entries directly (Stage-epsilon §2) — and runs the gates of §7 in continuous integration, rather than building either from nothing.

Revision log

Stage-delta — Gearing Dhar · v2.2 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0		First ratified edition.
v2.0		Named and fully specified the package manager as Kun (§3: manifest, lockfiles, versioning, workspaces, registry, security/provenance); added §4, Interop & the Foreign Ecosystem, covering C, C++, Rust, Java, and Python via Spec1's foreign keyword.
v2.1	September 7, 2026	§3.3 now requires lockfile reuse entirely from local cache with no network round-trip, per Casting Dhar's connectivity findings.
v2.2	October 26, 2026	Amendment A-1: the domain modules, in the order of the tracks, each with its gate (§2.1); numerics, tensors and quantum modules; test oracles are never dependencies (§3.8); acceptance criteria for the module gates (§7). Reference to Spec1's collections corrected to §13.